

SESSION 1

Classification Fundamentals

Logistic regression • Decision trees • Confusion matrices • Precision • Recall • F1-score

Applied AI Design Workshop for Faculty

Fred Livingston (fjliving@ncsu.edu) 2026

Goal: build intuition for predicting categories and choosing metrics based on the cost of errors.

Session focus

- Classification problem framing
- Probabilities and thresholds
- Logistic regression and trees
- Confusion matrices
- Precision, recall, and F1

Day 3 recap: foundations established

Session 1

Translated engineering and disciplinary problems into ML tasks; defined inputs, targets, units, baselines, and success criteria.

Session 2

Compared regression, classification, clustering, dimensionality reduction, forecasting, and anomaly detection.

Session 3

Introduced regression fundamentals, model baselines, residuals, and evaluation using MAE, RMSE, and R^2 .

Technical skills practiced

Loaded data with pandas, identified X and y, inspected DataFrames, visualized relationships, and used a basic scikit-learn regression workflow.

Day 4 transition

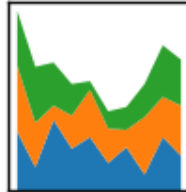
Move from continuous prediction to categorical prediction, then complete an end-to-end baseline workflow before data preparation and unsupervised discovery.

Opening prompt: Which Day 3 decision most affects whether a model is useful—problem framing, baseline selection, or metric choice?

Day 3 recap: Technical Skills

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



<https://pandas.pydata.org>

pandas is short for "PANel DAta S"

Pandas Paper: McKinney, Wes. "Data structures for statistical computing in python." *Proceedings of the 9th Python in Science Conference*. Vol. 445. 2010.

Data Wrangling

with pandas Cheat Sheet

<http://pandas.pydata.org>Pandas [API Reference](#) Pandas [User Guide](#)

Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = pd.MultiIndex.from_tuples(
        [('d', 1), ('d', 2),
         ('e', 2)], names=['n', 'v']))
```

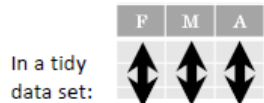
Create DataFrame with a MultiIndex

Method Chaining

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
      .rename(columns={
          'variable': 'var',
          'value': 'val'})
      .query('val >= 200'))
```

Tidy Data – A foundation for wrangling in pandas



In a tidy data set:

Each **variable** is saved in its own **column**Each **observation** is saved in its own **row**

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

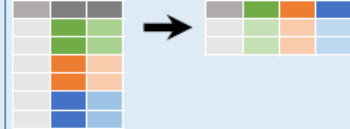


M * A

DAY 4
eringReshaping Data – Change layout, sorting, reindexing, renaming

```
pd.melt(df)
```

Gather columns into rows.



```
df.pivot(columns='var', values='val')
```

Spread rows into columns.



```
pd.concat([df1, df2])
```

Append rows of DataFrames



```
pd.concat([df1, df2], axis=1)
```

Append columns of DataFrames

```
df.sort_values('mpg')
```

Order rows by values of a column (low to high).

```
df.sort_values('mpg', ascending=False)
```

Order rows by values of a column (high to low).

```
df.rename(columns = {'y': 'year'})
```

Rename the columns of a DataFrame

```
df.sort_index()
```

Sort the index of a DataFrame

```
df.reset_index()
```

Reset index of DataFrame to row numbers, moving index to columns.

```
df.drop(columns=['Length', 'Height'])
```

Drop columns from DataFrame

Subset Observations - rows



```
df[df.Length > 7]
```

Extract rows that meet logical criteria.

```
df.drop_duplicates()
```

Remove duplicate rows (only considers columns).

```
df.sample(frac=0.5)
```

Randomly select fraction of rows.

```
df.sample(n=10)
```

Randomly select n rows.

```
df.nlargest(n, 'value')
```

Select and order top n entries.

```
df.nsmallest(n, 'value')
```

Select and order bottom n entries.

```
df.head(n)
```

Select first n rows.

```
df.tail(n)
```

Select last n rows.

Subset Variables - columns



```
df[['width', 'length', 'species']]
```

Select multiple columns with specific names.

```
df['width'] or df.width
```

Select single column with specific name.

```
df.filter(regex='regex')
```

Select columns whose name matches regular expression *regex*.

Using query

query() allows Boolean expressions for filtering rows.

```
df.query('Length > 7')
```

```
df.query('Length > 7 and Width < 8')
```

```
df.query('Name.str.startswith("abc")', engine="python")
```

Subsets - rows and columns

Use `df.loc[]` and `df.iloc[]` to select only rows, only columns or both.

Use `df.at[]` and `df.iat[]` to access a single value by row and column.

First index selects rows, second index columns.

```
df.iloc[10:20]
```

Select rows 10-20.

```
df.iloc[:, [1, 2, 5]]
```

Select columns in positions 1, 2 and 5 (first column is 0).

```
df.loc[:, 'x2': 'x4']
```

Select all columns between x2 and x4 (inclusive).

```
df.loc[df['a'] > 10, ['a', 'c']]
```

Select rows meeting logical condition, and only the specific columns.

```
df.iat[1, 2]
```

Access single value by index

```
df.at[4, 'A']
```

Access single value by label

Logic in Python (and pandas)

<	Less than	!=	Not equal to
>	Greater than	df.column.isin(values)	Group membership
==	Equals	pd.isnull(obj)	Is NaN
<=	Less than or equals	pd.notnull(obj)	Is not NaN
>=	Greater than or equals	&, , ~, ^, df.any(), df.all()	Logical and, or, not, xor, any, all

regex (Regular Expressions) Examples

\".	Matches strings containing a period '.'
'Length\$'	Matches strings ending with word 'Length'
^Sepal'	Matches strings beginning with the word 'Sepal'
^x[1-5]'	Matches strings beginning with 'x' and ending with 1,2,3,4,5
^(?!Species\$).*	Matches strings except the string 'Species'

Data Wrangling

with pandas Cheat Sheet

<http://pandas.pydata.org>Pandas [API Reference](#) Pandas [User Guide](#)

Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = pd.MultiIndex.from_tuples(
        [('d', 1), ('d', 2),
         ('e', 2)], names=['n', 'v']))
```

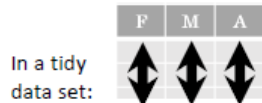
Create DataFrame with a MultiIndex

Method Chaining

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
      .rename(columns={
          'variable': 'var',
          'value': 'val'})
      .query('val >= 200'))
```

Tidy Data – A foundation for wrangling in pandas



In a tidy data set:

Each variable is saved in its own column



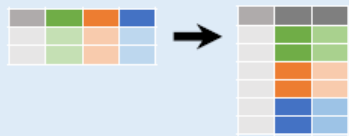
Each observation is saved in its own row

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.



M * A

Reshaping Data – Change layout, sorting, reindexing, renaming



```
pd.melt(df)
```

Gather columns into rows.



```
df.pivot(columns='var', values='val')
```

Spread rows into columns.



```
pd.concat([df1, df2])
```

Append rows of DataFrames



```
pd.concat([df1, df2], axis=1)
```

Append columns of DataFrames

```
df.sort_values('mpg')
```

Order rows by values of a column (low to high).

```
df.sort_values('mpg', ascending=False)
```

Order rows by values of a column (high to low).

```
df.rename(columns = {'y': 'year'})
```

Rename the columns of a DataFrame

```
df.sort_index()
```

Sort the index of a DataFrame

```
df.reset_index()
```

Reset index of DataFrame to row numbers, moving index to columns.

```
df.drop(columns=['Length', 'Height'])
```

Drop columns from DataFrame

Subset Observations - rows



```
df[df.Length > 7]
```

Extract rows that meet logical criteria.

```
df.drop_duplicates()
```

Remove duplicate rows (only considers columns).

```
df.sample(frac=0.5)
```

Randomly select fraction of rows.

```
df.sample(n=10)
```

Randomly select n rows.

```
df.nlargest(n, 'value')
```

Select and order top n entries.

```
df.nsmallest(n, 'value')
```

Select and order bottom n entries.

```
df.head(n)
```

Select first n rows.

```
df.tail(n)
```

Select last n rows.

Subset Variables - columns



```
df[['width', 'length', 'species']]
```

Select multiple columns with specific names.

```
df['width'] or df.width
```

Select single column with specific name.

```
df.filter(regex='regex')
```

Select columns whose name matches regular expression regex.

Using query

query() allows Boolean expressions for filtering rows.

```
df.query('Length > 7')
```

```
df.query('Length > 7 and Width < 8')
```

```
df.query('Name.str.startswith("abc")',
        engine="python")
```

Subsets - rows and columns

Use `df.loc[]` and `df.iloc[]` to select only rows, only columns or both.

Use `df.at[]` and `df.iat[]` to access a single value by row and column.

First index selects rows, second index columns.

```
df.iloc[10:20]
```

Select rows 10-20.

```
df.iloc[:, [1, 2, 5]]
```

Select columns in positions 1, 2 and 5 (first column is 0).

```
df.loc[:, 'x2': 'x4']
```

Select all columns between x2 and x4 (inclusive).

```
df.loc[df['a'] > 10, ['a', 'c']]
```

Select rows meeting logical condition, and only the specific columns.

```
df.iat[1, 2]
```

Access single value by index

```
df.at[4, 'A']
```

Access single value by label

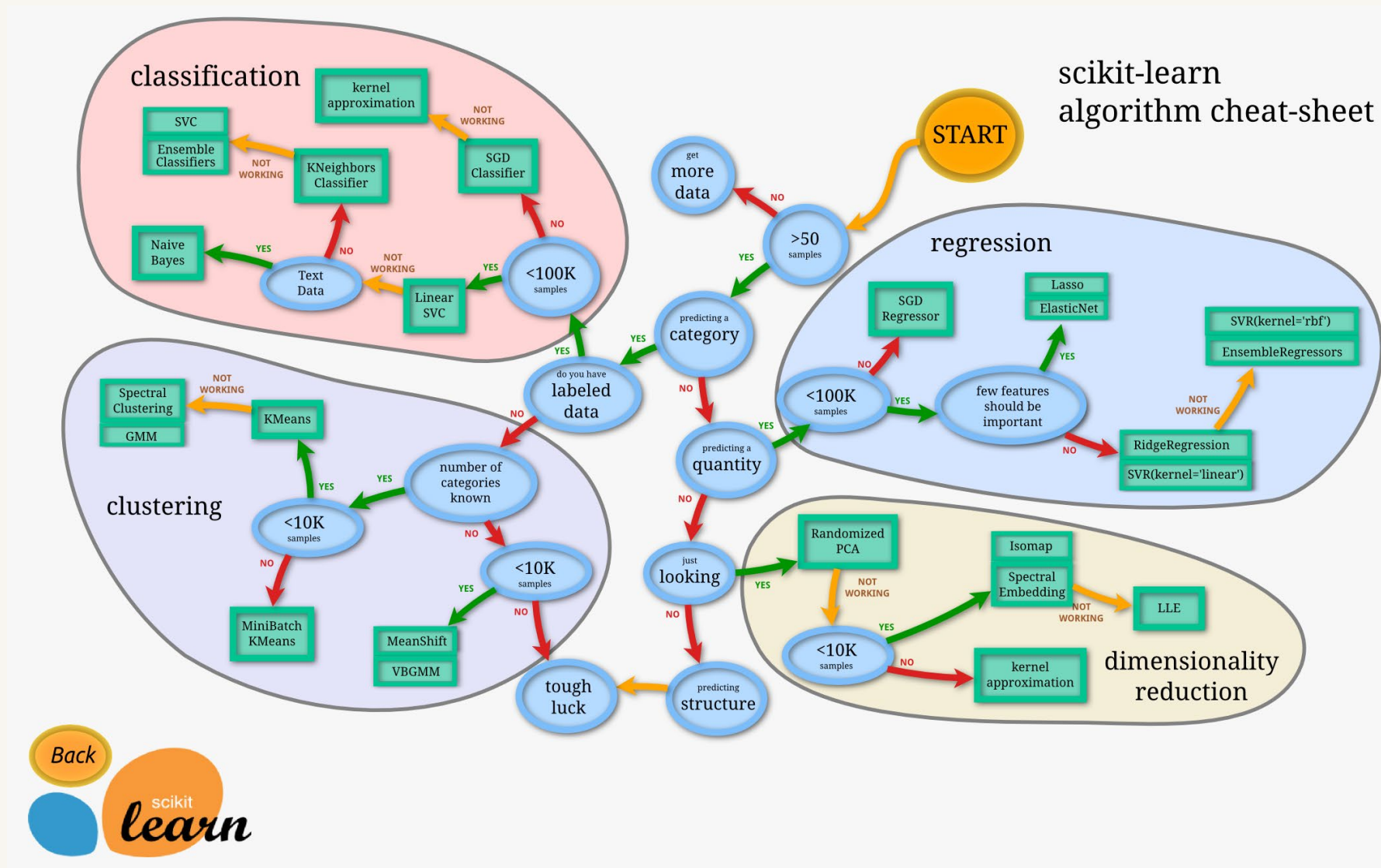
Logic in Python (and pandas)

<	Less than	!=	Not equal to
>	Greater than	df.column.isin(values)	Group membership
==	Equals	pd.isnull(obj)	Is NaN
<=	Less than or equals	pd.notnull(obj)	Is not NaN
>=	Greater than or equals	&, , ~, ^, df.any(), df.all()	Logical and, or, not, xor, any, all

regex (Regular Expressions) Examples

Regex	Example
'\.'	Matches strings containing a period '.'
'Length\$'	Matches strings ending with word 'Length'
'^Sepal'	Matches strings beginning with the word 'Sepal'
'^x[1-5]\$'	Matches strings beginning with 'x' and ending with 1,2,3,4,5
'^(?!Species)\$.*'	Matches strings except the string 'Species'

Day 3 recap: Technical Skills



Day 4 objectives and learning outcomes

Objective: extend the applied ML workflow from regression to classification, baseline modeling, data preparation, and unsupervised learning

Classify

Apply logistic regression and decision trees to categorical outcomes.

Prepare

Identify missing values, outliers, data-type problems, categorical variables, and leakage risks.

Evaluate

Interpret probabilities, confusion matrices, accuracy, precision, recall, and F1-score.

Engineer

Apply scaling, encoding, feature selection, and domain-informed feature development.

Compare

Evaluate candidate models against a simple majority-class or mean baseline.

Discover

Distinguish supervised from unsupervised learning and explain k-means, hierarchical clustering, and DBSCAN.

Build

Complete a basic regression or classification workflow from preparation through initial evaluation.

Interpret

Use disciplinary knowledge and validation evidence to judge whether clusters are meaningful.

Day 4 schedule

Session 1 — Classification Fundamentals

Session 2 — Hands-On Baseline ML Model

Break

Invited Presentation — LLM and AI Workflows on NCSU's Hazel HPC

Session 3 — Data Preparation and Feature Engineering

Special Topic Session

Session 4 — Unsupervised Learning for Discovery

Session 1 learning outcomes

Frame

Recognize when a categorical target makes classification appropriate

Explain

Describe logistic regression, decision trees, and predicted probabilities

Evaluate

Interpret confusion matrices and common classification metrics

Choose

Select metrics based on consequences of false positives and false negatives

Prepare

Connect classification concepts to the baseline model activity

Participant output: a classification problem statement, a baseline choice, an initial metric, and one discussion of error consequences.

What makes a problem a classification problem?

Classification predicts a category or class label. The model is judged by whether the predicted class is correct and which errors matter most.

Use classification when asking:

- Which category?
- Pass or fail?
- Normal or faulty?
- Eligible or not?
- Which condition?

Typical targets:

- Defective / acceptable
- Fault / no fault
- Risk level
- Material type
- Operating mode
- Course outcome

Not classification when:

- Target is continuous
- Goal is discovering unknown groups
- No labeled examples exist
- The output is a ranking only

Key discipline question: Which error is more costly — a false alarm or a missed case?

Activity: write a classification problem statement

Use the template to convert a disciplinary problem into a classification task.

Using _____ as inputs, develop a classification model to predict _____ for each _____, with performance evaluated using _____.

Inputs

Variables available
before prediction

Class label

Category or state to
predict

Unit

Part, sample, student,
machine cycle, image,
day

Metric

Accuracy, precision,
recall, F1, or class-
specific target

Quick check: are the labels trustworthy, consistent, and available for enough examples?

Example: product quality classification

A classification dataset pairs measured features with a categorical label such as acceptable or defective.

Part	Temp	Speed	Vibration	Roughness	Label
1	185	42	0.18	2.4	Accept
2	190	46	0.35	4.2	Defect
3	174	39	0.22	2.9	Accept
4	205	52	0.48	5.1	Defect
5	198	44	0.20	2.5	Accept

Prediction objective
Classify whether a part is acceptable before final inspection.

Baseline
Predict the majority class for every part.

Metric
Use recall for defects when missed defects are expensive.

Start with a classification baseline

A baseline clarifies whether the classifier adds value beyond the simplest possible decision rule.

Majority-class baseline

Predict the most frequent class for every case.

Essential when classes are imbalanced.

Random or stratified baseline

Predict classes based on observed class proportions.

Useful for sanity checking.

Domain-rule baseline

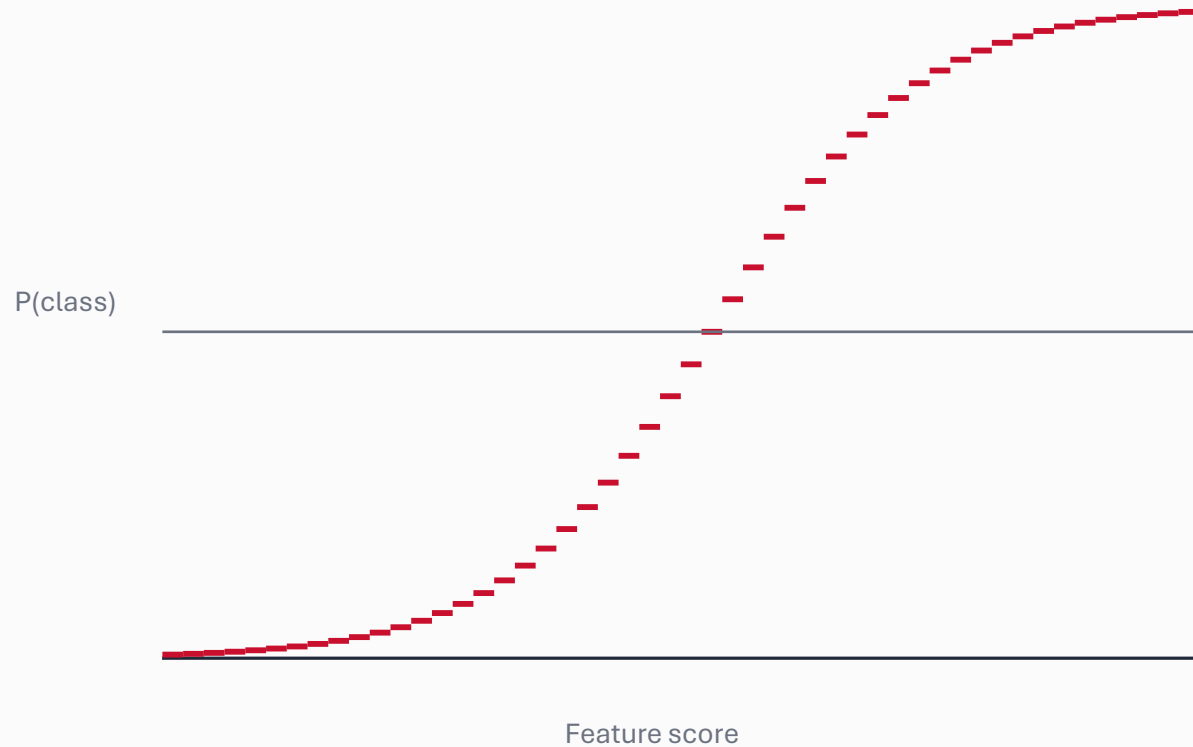
Use a threshold, checklist, expert rule, or existing procedure.

Often the fairest comparison.

Discussion prompt: Would stakeholders accept a model that beats accuracy but misses the most important cases?

Logistic regression: probabilities for class membership

Logistic regression estimates the probability that an observation belongs to a class, then applies a decision threshold.



Decision rule

If $P(\text{defect}) \geq \text{threshold} \rightarrow \text{classify as defect}$

Interpretation

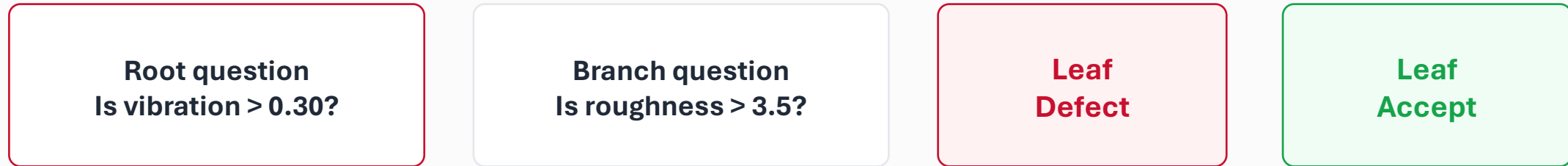
Coefficients describe how features shift log-odds, not direct class labels.

Caution

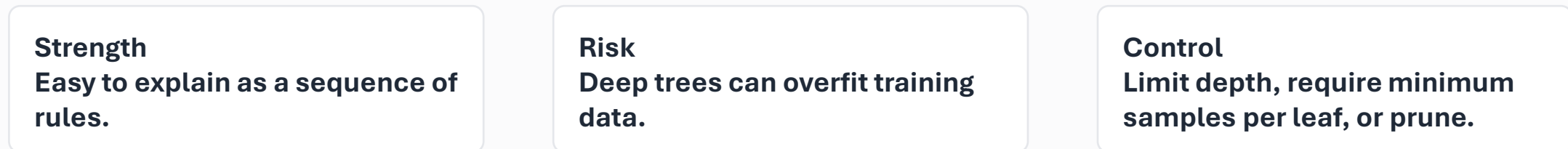
The default 0.50 threshold may not match the cost of errors.

Decision trees: interpretable rule-based classification

Decision trees split the data into increasingly pure groups using feature-based rules.



A path through the tree becomes a rule: if vibration is high, flag defect; otherwise inspect roughness.



Confusion matrix: the basic language of classification error

A confusion matrix separates correct predictions from different types of mistakes.

		Predicted class	
		Defect	Accept
Actual class	Defect	TP Detected defects	FN Missed defects
	Accept	FP False alarms	TN Correct accepts

False negative: a defect is missed.

False positive: an acceptable part is flagged.

Different applications assign different costs to these errors.

Classification metrics: accuracy is not enough

Metrics should match the consequences of different mistakes, especially for imbalanced classes.

Accuracy

Overall percent correct

Can be misleading when one class dominates

Precision

Of predicted positives, how many were correct?

Use when false alarms are costly

Recall

Of actual positives, how many were found?

Use when missed cases are costly

F1-score

Balance of precision and recall

Useful when both false alarms and misses matter

Metric choice is a design decision: it defines which mistakes the model is allowed to make.

Class imbalance: when accuracy hides failure

A model can appear highly accurate by ignoring rare but important classes.

Scenario

1000 parts
950 acceptable
50 defective

Majority baseline predicts
every part as acceptable.

Result

Accuracy = 95%
Defect recall = 0%

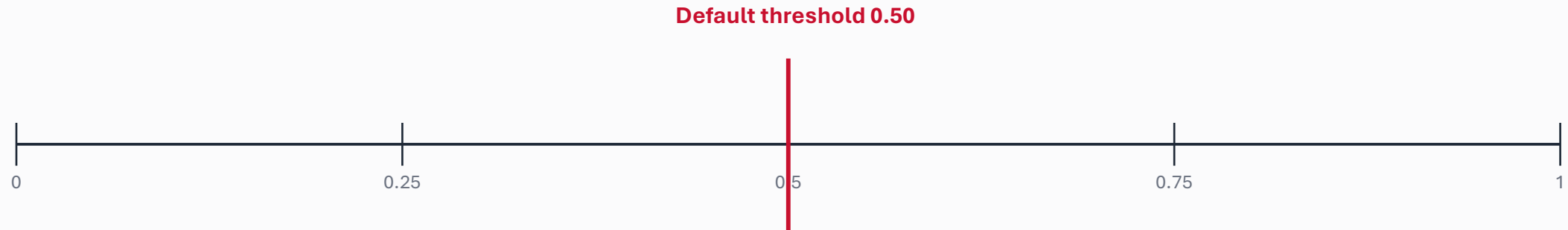
**The model misses every
defective part.**

Better questions

- How many defects are found?
- How many false alarms occur?
- What threshold should trigger review?

Decision thresholds: choosing where to draw the line

Many classifiers return probabilities. Changing the threshold changes false positives and false negatives.



Lower threshold
More positives flagged
Higher recall
More false alarms

Higher threshold
Fewer positives flagged
Higher precision
More missed cases

Best threshold
Depends on cost, capacity, risk,
and review process

Activity: interpret model outputs

Compare two classifiers and decide which is more appropriate for quality inspection.

Model A

Accuracy: 94%
Precision: 70%
Recall: 40%
F1-score: 51%

Model B

Accuracy: 91%
Precision: 55%
Recall: 82%
F1-score: 66%

Decision

**Which model should be
selected if missed defects are
costly?**

**Which if false alarms are
costly?**

Debrief: a lower accuracy model may be the better disciplinary choice when it reduces the more serious error.

Session 1 technical skill: minimal classification workflow

Goal: compare a majority-class baseline with logistic regression using classification metrics.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

X = df[features]
y = df["defect"]
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,
random_state=42)
```

```
baseline = np.full(len(y_test), y_train.mode()[0])
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
pred = model.predict(X_test)

recall = recall_score(y_test, pred)
f1 = f1_score(y_test, pred)
```

- Accuracy: overall proportion correct
- Precision: when model flags a defect, how often is it right?
- Recall: of all true defects, how many were found?
- F1-score: balance of precision and recall

[Session 1 notebook](#)[04_session4_classification_sklearn.ipynb](#)

Classification interpretation: inspect the confusion matrix

The preferred metric depends on the consequence of each error type.

	Predicted acceptable	Predicted defect
Actual acceptable	True Negative	False Positive
Actual defect	False Negative	True Positive

Instructor prompt

“Which error matters more in this application: a false alarm or a missed defect?”

- Accuracy can be misleading when defects are rare.
- Recall is critical when missed defects are expensive.
- Precision is critical when false alarms are expensive.

Classroom discussion prompts and transition

Why can accuracy be misleading?

Which error is more costly: false positive or false negative?

Should the threshold always be 0.50?

When is a decision tree preferable to logistic regression?

How trustworthy are the class labels?

What baseline represents current practice?

Session synthesis: classification is not just about predicting labels — it is about choosing errors that align with disciplinary consequences.

Next: Hands-on baseline model activity — apply regression or classification to a small dataset.